MODULE 36 · DOMAIN 4 - RESILIENCE, MIGRATION AND CONTINUITY

Mainframe and Legacy Modernisation

Understand the code before changing it, choose between replatform, refactor and reimagine, prove equivalence with tests, and modernise in increments.

DOMAIN 4 of 5 EXAM WEIGHT 15% MODULE 36 of 52

admin@sthithapragna.com

The hardest part of a legacy system is not the platform. It is the **business logic nobody can explain** any more.

Mainframe modernisation paths

PATH	WHAT CHANGES	FITS
Replatform	COBOL kept, emulating runtime	Fast exit, COBOL skills
Refactor	COBOL to Java, automatically	Logic preserved, Java teams
Reimagine	Rules extracted, new design	Batch to real-time
Replace	Commercial or SaaS product	Non-differentiating function
Augment	Data and APIs extended to AWS	Mainframe staying for now

ON THE EXAM

Speed with least change: replatform. Logic preserved in Java: refactor. New architecture: reimagine.

AWS Transform for mainframe

STAGE	WHAT THE AGENTS DO
Analyse	Inventory COBOL, PL/I, JCL, CICS, BMS, Db2, VSAM, IMS
Document	Generate technical documentation and business rules
Decompose	Group code into business domains, approved by people
Plan	Order domains into modernisation waves
Refactor or reimagine	Java with logic preserved, or new specs and services
Test	Test plans, data collection scripts, automated functional tests

ON THE EXAM

Every stage has humans approving the output: decomposition, waves, specifications.

Refactor or reimagine?

REFACTOR

Same behaviour, new language

- COBOL to Java, logic preserved
- Same batch and transaction structure
- Functional equivalence is the goal
- Faster, lower risk

REIMAGINE

Same rules, new architecture

- Reverse-engineer business rules to specs
- Forward-engineer microservices and data model
- Batch can become real-time
- Slower, more validation by experts

THE TIEBREAKER

Keep how it works, change the language: refactor. Keep what it decides, change how it works: reimagine.

Proving equivalence

- **Test plans** generated from code, business logic and scheduler plans.
- **Data collection scripts** capture real inputs from VSAM and Db2.
- **Functional tests** run the same cases against the new application.
- **Parallel runs** compare outputs before the old system is switched off.
- **Batch order matters**: tests follow the JCL sequence.

> ON THE EXAM
>
> No equivalence evidence, no cutover. Automated test generation is part of the answer.

What changed in the mainframe service

OFFERING	STATUS FOR NEW CUSTOMERS
Managed runtime environment (Rocket)	Closed; use the self-managed runtime
Self-managed partner capabilities	Closed; vendor-direct or AWS Transform
Self-managed runtime for Rocket replatform	Available
Runtime for AWS Transform refactored code	Available
AWS Transform for mainframe	Available

ON THE EXAM

An answer offering the managed Mainframe Modernization runtime to a new customer is stale.

Modernising in increments

- **Domains move one at a time**, in the waves AWS Transform proposes.
- **APIs in front**: mainframe transactions exposed through API Gateway.
- **Route by route**: switch an API to the modernised service when ready.
- **Data in step**: CDC keeps Db2 and the new database synchronised.
- **Analytics first**: offload reporting to AWS before moving transactions.

> ON THE EXAM
>
> The strangler fig works for mainframes too: APIs in front, domains moved behind them.

Reading the scenario

WHEN THE QUESTION SAYS	IT IS TESTING
COBOL to Java, logic preserved	AWS Transform refactor
Batch to real-time, new architecture	AWS Transform reimagine
Exit hardware fast, keep COBOL	Replatform on self-managed runtime
Mainframe stays; cloud needs its data	CDC replication to AWS
Prove the new system matches	Generated test plans and data
Move transactions gradually	APIs in front, strangler fig

ON THE EXAM

Understand first, choose the path by how much must change, then prove equivalence.

> Meridian Freight's billing runs on 2 million lines of COBOL with CICS and Db2. The last COBOL developers retire next year, and the company's teams work in Java.
>
> The business logic must be preserved exactly, and the team wants as much automation as possible.

What should the solutions architect recommend?

A Rewrite the application in Java by hand from interviews

B AWS Transform for mainframe to analyse, decompose and refactor the COBOL to Java

C Replatform the COBOL onto an emulating runtime

D Buy a SaaS billing product

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Manual rewrite

Slow, and logic is lost when nobody can explain it.

B AWS Transform refactor

Automated analysis and COBOL-to-Java with logic preserved.

C Replatform

Keeps COBOL, which nobody will be able to maintain.

D SaaS

Does not preserve Meridian's specific billing logic.

THE SIGNAL

Skills leaving, Java teams, logic preserved, automated: refactor with AWS Transform.

> Meridian Freight's mainframe calculates customs duties in an overnight batch, so customers wait until the next day for a quote. The business wants instant quotes.
>
> The duty rules themselves must stay the same.

Which modernisation approach fits best?

A Replatform the batch to an emulating runtime on AWS

B Refactor the batch COBOL to Java batch jobs

C Reimagine with AWS Transform: extract the duty rules, then build them as real-time services

D Run the batch every hour instead of nightly

◆ PAUSE AND DECIDE

Which modernisation approach fits best?

A Replatform
Still an overnight batch.

B Refactor
Same batch structure, just in Java.

C Reimagine
Same rules, new real-time architecture.

D Hourly batch
Faster, but still not instant.

THE SIGNAL

Same rules, different architecture: reimagine.

> Meridian Freight's mainframe hardware lease ends in nine months. The operations team has deep COBOL skills, and the company has no budget to change application code this year.
>
> Meridian has never used AWS Mainframe Modernization.

What should the solutions architect recommend?

A. Replatform onto the self-managed Rocket runtime from AWS Mainframe Modernization on EC2

B. The AWS Mainframe Modernization managed runtime environment

C. Reimagine the whole estate as microservices

D. Sign a new five-year mainframe lease

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Self-managed replatform

COBOL unchanged, available to new customers, fits nine months.

B Managed runtime

No longer open to new customers.

C Reimagine

Far too long, and changes code.

D New lease

Abandons the migration.

THE SIGNAL

Fast exit, no code change, new customer: the self-managed replatform runtime.

> Meridian Freight will keep its mainframe for at least three more years. The analytics team wants shipment data from Db2 for z/OS in AWS within minutes of each change.
>
> Running analytical queries on the mainframe is expensive and slows transactions.

What should the solutions architect recommend?

A Nightly file extracts sent by FTP to S3

B Run the analytics queries directly against Db2 from AWS

C Migrate the whole mainframe first

D Change data capture replication from Db2 for z/OS into AWS data stores

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Nightly FTP

A day old, not minutes.

B Direct queries

The expensive mainframe processing the business wants to avoid.

C Migrate first

Years away; the mainframe is staying.

D CDC replication

Changes reach AWS in minutes; analytics run off the mainframe.

THE SIGNAL

Mainframe stays, cloud needs fresh data: augment with change data capture.

> Meridian Freight has refactored a batch settlement application from COBOL to Java. Auditors require evidence that every batch job produces identical results to the mainframe before cutover.
>
> The batch jobs run in a strict scheduler sequence.

What should the solutions architect recommend?

A. Rely on the Java unit tests produced during refactoring

B. Use AWS Transform's generated test plans, data collection scripts and automated functional tests, following the scheduler sequence

C. Ask users to check a sample of reports after cutover

D. Compare row counts between the two databases once

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Unit tests

Test code units, not end-to-end equivalence with the mainframe.

B Generated functional tests

Real data, same cases, same sequence, repeatable evidence.

C User sampling after cutover

Too late, and not evidence for every job.

D Row counts

Equal counts can hide different values.

THE SIGNAL

Equivalence evidence for every batch job, in sequence, before cutover: generated functional tests.

> Meridian Freight's mainframe handles booking, tracking and invoicing as CICS transactions called by web and partner systems. AWS Transform has decomposed the code into these three domains.
>
> The company wants to move one domain at a time while the others keep running on the mainframe.

What should the solutions architect implement?

A. A single weekend cutover of all three domains

B. Run a full copy of the system on AWS in parallel for two years

C. Expose the transactions as APIs through API Gateway, switch each domain's routes to its modernised service when ready, and keep data in step with CDC

D. Freeze all mainframe changes until modernisation completes

◆ PAUSE AND DECIDE

What should the solutions architect implement?

A Weekend cutover

The big bang the company is avoiding.

B Two-year parallel run

Doubles cost and effort without incremental value.

C APIs, route switching, CDC

The strangler fig, domain by domain.

D Change freeze

Stops the business for the length of the programme.

THE SIGNAL

One domain at a time behind stable interfaces: the strangler fig for mainframes.

What to carry out of this module

- **Understand first**: analysis, documentation, decomposition into domains.
- **Replatform** for speed, **refactor** to Java, **reimagine** for new architecture.
- **Prove equivalence** with generated tests on real data.
- **Managed runtime is closed** to new customers; use self-managed.
- **Augment and strangle**: CDC for data, APIs for gradual moves.

> ON THE EXAM
>
> Understand, choose how much to change, prove it, move domain by domain.